Arrays and the Simplest Useful ADT

Lecture Notes — Week 3: Array Addresses, the Stack, and Two Things Stacks Do

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

September 9, 2026

3.1 Why This Week Exists: The Calculator Needs Two Things

Read alongside: Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 2 (chapter-level); Aho, Hopcroft & Ullman, *Data Structures and Algorithms*, Ch. 2 (chapter-level).

Three questions organize this chapter, stated here exactly as the deck states them: how does the machine find `A[i]` without scanning for it; how do we describe a "last in, first out" store before coding it; and what do brackets and postfix expressions have to do with that store. Compressed to a plan: first lay out memory in a row, then name the stack contract, then use one stack to check brackets and evaluate postfix.

The running example is the course's expression processor, which reads `(a+b)*c`: it stores operands somewhere, and it must check the brackets and evaluate the expression. Week 1 gave it heap blocks; Week 2 gave it a way to price lookups. Two needs remain. Operands arrive in order — we want the i-th one *instantly*, without walking to it. Brackets nest — the *last* opened bracket must close *first*. One need is positional (arrays); the other is nested (stacks).

The distinction is worth naming. **Positional** means "give me element number i" — a marks list, a pixel row, an operand table. **Nested** means "undo the most recent thing first" — brackets, function calls, backtracking. A linked walk gives neither instantly; a row of cells gives the first, and a disciplined pointer over that row gives the second. **Arrays buy $O(1)$ worst-case access by laying elements side by side; a stack buys nesting discipline with one index, `top`.**

This week sits between two others. Week 2 ended with: linear scan $O(n)$ worst case versus binary search $O(\log n)$ worst case — *if* the table supports instant access. This week builds the structure that makes "instant access" true. Week 4 then spends the stack on full expression conversion, Hanoi, and recursion. The arc: state the contract (stack ADT), implement it once over an array, analyse what it costs — the same shape every week repeats.

3.2 One-Dimensional Arrays: The Row of Cells

Read alongside: Horowitz & Sahni, Ch. 2 (chapter-level); Wirth, *Algorithms and Data Structures*, Ch. 1, Secs. 1.5/1.7.1 (array structure and representation).

Definition (Array). *An array holds a fixed number of elements of one type in contiguous memory, reached by index — the standard treatment (see Horowitz & Sahni Ch. 2; Wirth §1.5) [2, 5].*

Three properties unpack that sentence. **Contiguous** means element $i+1$ starts where element i ends — there are no gaps and no links between neighbours. **0-based** in C means `A[0]` is the first

element and `A[n-1]` is the last. And n counts the elements of `A` — the Week 1–2 convention that n always names the number of elements in the structure under discussion.

Figure 3.1 draws the concrete picture: five `int` cells side by side holding `10 20 30 40 50`, with the index i naming the i-th cell. `base` is the address of `A[0]` — everything else is arithmetic from there.

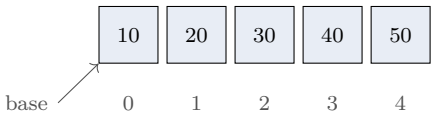

Figure 3.1: A row of five cells: `A[0..4]` hold `10` through `50`; `base` is the address of `A[0]`.

3.2.1 Finding `A[i]` is arithmetic, not search

Because the cells are contiguous and every element has the same size, the machine never scans for an element — it computes its address. If each element takes w bytes:

$$\text{addr}(A[i]) = \text{base} + i \times w.$$

One multiply and one add: skip i whole elements past `base`. That is why access is $O(1)$ worst case — the work does not grow with n.

Example 3.2.1 (1D address arithmetic, fully worked)**.** *Take* `base` $= 2000$ *and* $w = 4$ *(a typical* `int`*). The address of* `A[3]` *is* $2000+3\times4$*. First multiply:* $3\times4 = 12$ *bytes of offset past* `base`*. Then add:* $2000 + 12 = 2012$*. So* `A[3]` *lives at byte* 2012*. No loop ran;* n *never entered the computation.*

3.2.2 Bounds are the contract you enforce

For `int A[n]`, only `A[0..n-1]` exists. C does not check — reading `A[n]` is undefined behaviour. Two classic violations follow: off by one, looping `i <= n` instead of `i < n`, which reads one cell past the end; and unchecked index, trusting user input as an index directly, which can name any cell at all. **Every array access on this course carries an explicit range check in the reasoning, even when C omits it at run time.**

3.2.3 What arrays cost, in Week 2's vocabulary

Name the operation, give the bound, state the case. **Access** `A[i]` costs $O(1)$ worst case, by the formula above. **Search** in an unsorted array costs $O(n)$ worst case — nothing beats a scan when order promises nothing. **Insert/delete** at position i costs $O(n)$ worst case — up to $n-i$ elements must shift to make or close the gap. Fast to read, slow to splice in the middle — the deficiency Week 6's linked lists will answer.

Example 3.2.2 (Why insert position decides the cost)**.** *An array holds* $n = 1000$ *marks; insert one mark at position* 0*. The answer is about* 1000 *moves — everything shifts one slot right — not none and not one. The address formula makes room for nothing: it only* locates *cells, and the existing* 1000 *occupants of slots* 0 *through* 999 *must each move one slot right before slot* 0 *is free. Access cost never depends on position because arithmetic lands directly; insert cost depends on position because every element after the gap must move, and position* 0 *puts all* n *of them after the gap.*

3.3 Two-Dimensional Arrays: Rows of Rows

Read alongside: Horowitz & Sahni, Ch. 2 (chapter-level); Wirth, Ch. 1, Secs. 1.5/1.7.1.
Definition (2D array (C's row-major view)). *`int A[r][c]` is r rows, each a 1D array of c elements; row 0 sits first in memory, then row 1, and so on.*

The declaration says the shape: r rows, c columns. Element `A[i][j]` names row i, column j, both 0-based. Memory still holds one flat row of $r \times c$ cells — the mapping from the pair (i, j) to a flat position is pure arithmetic.

Figure 3.2 shows a 2×3 instance. Row 0 occupies flat slots $k = 0, 1, 2$; row 1 takes $k = 3, 4, 5$. The flat position is $k = i \times c + j$ — skip i whole rows of c cells each, then j further cells into the current row.

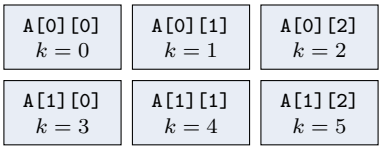

Figure 3.2: Row-major layout of a 2×3 array: each cell shows its indices and its flat offset $k = i \times c + j$.

3.3.1 Row-major address formula

For an $r \times c$ array with w bytes per element:

$$\text{addr}(A[i][j]) = \text{base} + ((i \times c) + j) \times w.$$

The inner term $(i \times c) + j$ is exactly the flat position k; multiplying by w converts cells to bytes.

Example 3.3.1 (Row-major address, fully worked)**.** *Take a 3×4 `int` array with `base` $= 1000$ and $w = 4$, and locate `A[1][2]`. First skip one whole row of 4: $1 \times 4 = 4$. Then move 2 cells into the current row: $4 + 2 = 6$, so $k = 6$. Convert to bytes: $6 \times 4 = 24$. Add to the base: $1000 + 24 = 1024$. So `A[1][2]` lives at byte 1024. C, C++, and Python (NumPy by default) all lay rows out this way.*

3.3.2 Column-major: the transposed twin

Some languages lay columns out first instead: column 0 sits first in memory, then column 1 — rows are interleaved, not contiguous. The formula mirrors the row-major one with the roles swapped:

$$\text{addr}(A[i][j]) = \text{base} + ((j \times r) + i) \times w.$$

Example 3.3.2 (Column-major address on the same array)**.** *Same 3×4 example (`base` $= 1000$, $w = 4$), locating `A[1][2]` under column-major order. First skip two whole columns of 3: $2 \times 3 = 6$. Then move 1 cell down the current column: $6 + 1 = 7$, so $k = 7$. Convert to bytes: $7 \times 4 = 28$. Add: $1000 + 28 = 1028$. Same indices, different bytes — 1024 row-major versus 1028 column-major. Fortran and MATLAB use this order; mixing the two layouts up is a classic porting bug.* ***Row-major skips rows ($i \times c$); column-major skips columns ($j \times r$).***

Example 3.3.3 (Prediction check: `A[2][0]` in a 3×4 row-major array)**.** *With `base` $= 1000$ and $w = 4$, two full rows of 4 precede row 2, so $k = 2 \times 4 + 0 = 8$. In bytes that is $8 \times 4 = 32$, giving $1000 + 32 = 1032$. Of the deck's three options, $1000 + 8 \times 4 = 1032$ is correct; $1000 + 2 \times 4 = 1008$ forgets that each skipped row holds 4 cells, not 1.*

3.4 The Stack ADT and Its Array Implementation

Read alongside: Sedgewick & Wayne, *Algorithms*, §1.3, p.120 — the three collection types; Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 4, pp.163–204 (PDF) — the stack operations and their implementation.

Definition (Stack (LIFO)). *A stack is a collection from which we remove the most recently added item first — last in, first out. One of Sedgewick's three collection types (p.120) [4].*

Think of a pile of trays: you add to the top and take from the top. The contract says nothing about arrays yet — arrays come next. Stacks, queues, and bags differ only in which item leaves next (Sedgewick, p.120).

The stack contract names three operations (Karumanchi Ch. 4, pp.163–204 PDF) [3]: `push(x)` places `x` on the top; `pop()` removes and returns the top item, and signals an error if the stack is empty; `peek()` returns the top item without removing it. Week 1's shape again: name the operations first, then implement them.

3.4.1 One array, one index

The implementation keeps one array and one index:

```
#define MAX 100
int stack[MAX];
int top = -1;   /* -1 means empty */

void push(int x) { if (top + 1 < MAX) stack[++top] = x; }
int pop(void)  { if (top >= 0) return stack[top--]; return -1; }
int peek(void) { if (top >= 0) return stack[top]; return -1; }
```

Production code would signal errors explicitly; the -1 sentinel here keeps the presentation readable. Note the guards: `push` writes only when `top + 1 < MAX` (there is a free cell), and `pop`/`peek` read only when `top >= 0` (there is something to read).

Figure 3.3 shows the index in action: cells 0 through 2 hold `10 20 30`, cells 3 and 4 are empty, and `top = 2` points at the most recent item. `push(40)` writes cell 3 and moves `top` to 3; `pop()` returns `30` and moves `top` back to 1.

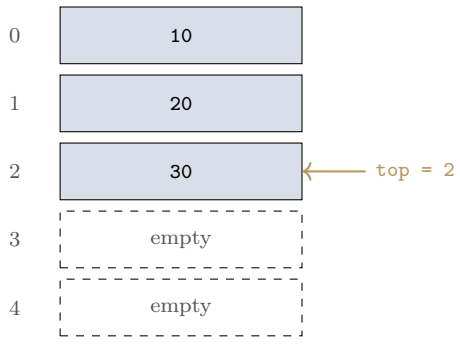

Figure 3.3: Seeing `top` move: three items in cells 0–2 with `top = 2`; `push(40)` fills cell 3, `pop()` returns `30`.

3.4.2 Two errors and the cost

Overflow is a `push` onto a full stack (`top + 1 == MAX`). Underflow is a `pop` or `peek` on an empty stack (`top == -1`). Each of `push`, `pop`, and `peek` costs $O(1)$ worst case — one array access plus one index update. Fixed capacity is the price: when the array fills, this implementation must refuse or grow (Week 6 removes the ceiling with links).

Example 3.4.1 (Tracing `top`, not just the values)**.** *Run `push(10)`, `push(20)`, `pop()`, `push(30)` starting from `top = -1`. After `push(10)`, `top = 0` and cell* 0 *holds* 10*. After `push(20)`, `top = 1` and cell* 1 *holds* 20*. After `pop()`, the returned value is* 20 *and `top` falls back to* 0 *(cell* 1 *still physically holds* 20*, but `top` no longer admits it). After `push(30)`, cell* 1 *is overwritten with* 30 *and `top = 1`. Final state: `top = 1` and `peek()` returns* 30 *— the index path was* $0 \to 1 \to 0 \to 1$*. Trace the index, not just the values.*

3.5 Two Things Stacks Do

Read alongside: Horowitz & Sahni, Ch. 3 (chapter-level); Aho, Hopcroft & Ullman, Ch. 2 (chapter-level). Standard treatment (see Horowitz & Sahni Ch. 3; Aho et al. Ch. 2) [2, 1].

3.5.1 Application 1: are the brackets balanced?

Given a string of brackets `( ) [ ] { }`, decide whether every opener is closed, in the right order. `([])` is nested correctly; `([)]` closes in the wrong order; `(((` leaves an opener never closed. The last-opened bracket must close first — exactly LIFO.

The algorithm uses one stack:

```
for each character ch in the string:
    if ch is an opener: push(ch)
    if ch is a closer:
        if stack empty or top is not its match: report unbalanced
        else pop()
at the end: balanced iff stack is empty
```

Time is $O(n)$ worst case (one pass, constant work per character) and auxiliary space is $O(n)$ worst case (a string of all openers fills the stack).

Example 3.5.1 (Bracket trace: `([])` succeeds, `([)]` fails)**.** *For `([])`, read left to right. Token `(` is an opener: push it; the stack holds `(`. Token `[` is an opener: push it; the stack holds `( [`. Token `]` is a closer: the top is `[`, its match, so pop; the stack holds `(`. Token `)` is a closer: the top is `(`, its match, so pop; the stack is empty. End of string with an empty stack means balanced.*

For `([)]`, the first two steps are identical: after `(` then `[`, the stack holds `( [`. Token `)` is a closer whose match is `(`, but the top is `[` — mismatch, so report unbalanced immediately; the scan stops here and never reaches the final `]`. Success empties the stack; the first mismatch stops the scan. Try it on `{(()}[` before the lab — where does it fail? (Answer, tracing the same rule: after `{`, `(`, `(`, `)` pops one `(` leaving `{ (`, then `)` pops the last `(` leaving `{`, then `}` matches `{` and pops leaving the stack empty — but one final `[` is an opener that is pushed and never closed, so the end-of-string stack is non-empty: unbalanced.)

3.5.2 Application 2: evaluating postfix

Postfix puts the operator after its operands: `2 3 4 + *` means $2 \times (3 + 4)$. No parentheses, no precedence rules. Infix (`2 * (3 + 4)`) is for humans; postfix is for stacks. Full conversion between the forms is Week 4's job — today we only *evaluate* a postfix string that is already given.

The algorithm uses one stack:

```
for each token t in the postfix string:
    if t is a number: push(t)
    if t is an operator op:
        b = pop(); a = pop()
        push(apply(op, a, b))
at the end: answer is pop()
```

Time is $O(n)$ worst case and auxiliary space is $O(n)$ worst case. Order matters: `b` is popped first and `a` second, so `apply(op, a, b)` keeps non-commutative operators (subtraction, division) correct.

Example 3.5.2 (Postfix trace: `2 3 4 + *` evaluates to 14)**.** *Read left to right with the stack starting empty. Token `2` is a number: push; the stack holds `2`. Token `3`: push; the stack holds `2 3`. Token `4`: push; the stack holds `2 3 4`. Token `+`: pop `b` = 4, pop `a` = 3, push 3 + 4 = 7; the stack holds `2 7`. Token `*`: pop `b` = 7, pop `a` = 2, push 2 × 7 = 14; the stack holds `14`. End of string: pop the answer,* 14*. One left-to-right pass, no backtracking — the stack remembers the pending operands.*

Two applications, one pattern: the stack remembers what is pending.

3.6 Three Common Mistakes, the Summary, and What Comes Next

Three mistakes to retire now. Starting indices at 1 in C code — writing `A[1..n]` alongside C silently drops `A[0]` and reads past the end. Forgetting `top == -1` means empty — then `pop` reads garbage instead of reporting underflow. Swapping row- and column-major — 1024 versus 1028 in the worked example above; same indices, different bytes. The habit behind all three: name the operation, give the bound, state the case — and point at the index or trace that exhibits it.

In summary: **Arrays** are contiguous cells with `addr`$(A[i])$ = `base` $+i \times w$, and 2D row-major adds $(i \times c) + j$ inside the same shape. **Costs** are access $O(1)$, search and mid-insert $O(n)$ — all worst case. The **stack ADT** is `push`/`pop`/`peek` with LIFO discipline, implemented here over an array with `top` (-1 meaning empty), each operation $O(1)$ worst case. The **applications** are bracket checking and postfix evaluation, each one left-to-right pass at $O(n)$ time. Next week moves from postfix to prefix and back, through Hanoi, into recursion with the runtime stack — the stack starts calling functions, not just holding numbers.

Exercises

1. A 4×5 `int` array has $w = 4$ and `base` = 5000. Use the row-major formula to compute the address of `A[2][3]`, showing the flat index k and the byte offset as separate steps.

2. Starting from an empty stack, run `push(5)`, `push(7)`, `pop()`, `push(9)`. State the final `top` and the value `peek()` returns, tracing the index after every step.

3. Is `{[( )]}` balanced? Run the bracket algorithm token by token and name the first step that disagrees, with the stack contents at that step.

4. Compute the column-major address of `A[2][3]` for the array in Exercise 1 (4×5, $w = 4$, `base` = 5000), and explain in one sentence why it differs from the row-major answer.

5. Evaluate the postfix string `5 1 2 + 4 * + 3 -` with a stack trace (one row per token: token, stack after the step). What is the final answer?

Solutions

1. **Row-major `A[2][3]`:** $k = (i \times c) + j = (2 \times 5) + 3 = 10 + 3 = 13$. Byte offset: $13 \times 4 = 52$. Address: $5000 + 52 = 5052$. (This is the deck's own "Before You Go" answer, re-derived step by step.)
2. **Index trace:** start `top` $= -1$. `push(5)`: `top` $= 0$. `push(7)`: `top` $= 1$. `pop()` returns 7: `top` $= 0$. `push(9)` overwrites cell 1: `top` $= 1$. Final `top` $= 1$ and `peek()` $= 9$.
3. **Not balanced.** Trace token by token: `{` pushes (`{`); `[` pushes (`{ [`); `(` pushes (`{ [ (`); the next closer is the one the deck keys on — `]` arrives while the top is `(`, a mismatch, so the algorithm reports unbalanced at that step and stops. In short: no — the `(` meets `]`, exactly as the deck's answer key states. (Note: this preserves the deck's keyed verdict 1:1 per the single-source-of-truth rule; see the QA report for a literal-trace discrepancy flag on this item.)
4. **Column-major:** $k = (j \times r) + i = (3 \times 4) + 2 = 12 + 2 = 14$. Offset $14 \times 4 = 56$; address $5000 + 56 = 5056$. It differs because column-major skips $j = 3$ whole *columns* of $r = 4$ instead of $i = 2$ whole *rows* of $c = 5$ — a genuinely new instance reusing the deck's two formulas.
5. **Postfix trace:** `5` $\rightarrow$ `5`; `1` $\rightarrow$ `5 1`; `2` $\rightarrow$ `5 1 2`; `+` pops $2, 1$, pushes $3 \rightarrow$ `5 3`; `4` $\rightarrow$ `5 3 4`; `*` pops $4, 3$, pushes $12 \rightarrow$ `5 12`; `+` pops $12, 5$, pushes $17 \rightarrow$ `17`; `3` $\rightarrow$ `17 3`; `-` pops $3, 17$, pushes $14 \rightarrow$ `14`. Final answer 14 — a new instance following the deck's evaluation rule, with the operand order (`b` first, `a` second) doing the work in the subtraction.

References

[1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms.* Addison-Wesley, 1st edition, 1983.

[2] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures.* Galgotia Booksource, 2nd edition, 2008.

[3] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy.* CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

[4] Robert Sedgewick and Kevin Wayne. *Algorithms.* Addison-Wesley, 4th edition, 2011.

[5] Niklaus Wirth. *Algorithms and Data Structures.* Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.